

Mutual Fund Order Management System

Project 3 Proposal | Team 29 | Domain: Finance

S26CS6.401 Software Engineering | Submission: 13 Mar 2026

1. Description of the Use Case

Problem Statement

Wealth Management divisions process thousands of mutual fund orders daily from clients, advisors, and automated portfolio systems. Current workflows are fragmented and manual-intensive, with no unified platform to handle ingestion, validation, aggregation, and transmission to transfer agents.

This results in duplicate order risks, missed transmission cutoffs, costly per-order fees, and poor lifecycle visibility, all of which carry direct financial and regulatory consequences.

Target Stakeholders

- Clients placing buy/sell orders via digital platforms
- Financial Advisors using web and mobile portals
- Portfolio Managers via automated PMS feeds
- Operations Teams managing exceptions and reconciliation
- Compliance and Audit teams requiring full order trails

Why This Matters

Mutual fund orders are time-sensitive and regulated. Missed transmission windows (NSCC at 4:00 PM ET, RBC at 1:00 AM ET) directly impact trade-date pricing and client portfolio accuracy. Without aggregation, each individual order incurs a separate transfer agent fee, multiplied across thousands of daily trades.

Duplicate orders or reconciliation failures can trigger regulatory scrutiny and financial liability under SEC and FINRA rules.

Solution Summary

A centralized platform that ingests orders from all channels, validates and enriches them, aggregates at the fund level into bulk orders, transmits to the correct transfer agent at the correct time, processes NAV contracts, and books trades into client accounts with real-time status notifications throughout.

2. Key Functionalities

Core Features

- Multi-channel order ingestion (REST API and batch)
- Validation: fields, eligibility, cutoffs, duplicates
- Fund enrichment with fund family and TA mapping
- Bulk order aggregation by fund and trade date
- Scheduled transmission to NSCC and RBC
- NAV contract processing and share allocation
- Trade booking and client position updates
- Real-time status notifications to all parties
- Exception management tools for operations
- Reconciliation dashboard with break alerting

User Interaction

- Advisor/Ops: React web portal
- Clients: mobile-friendly web app for order status
- PMS: REST and batch API endpoints

Tech Stack

- Backend: Python (FastAPI) and Node.js microservices
- Message Queue: Apache Kafka for async order streams
- Database: PostgreSQL (orders), Redis (dedup/idempotency)
- Frontend: React.js (advisor portal and ops dashboard)
- Scheduler: Apache Airflow for transmission windows
- Containerisation: Docker and Kubernetes
- API Gateway: Kong
- Monitoring: Grafana and Prometheus

Architectural Tactics

- Exactly-once processing via idempotency keys and Redis dedup at every stage
- Event-driven architecture using Kafka topics per order lifecycle stage
- CQRS separating order write path from reporting read path
- Circuit Breaker on transfer agent API calls with retry backoff
- Order immutability enforcement after transmission

Design Patterns

- Saga Pattern for distributed transaction management across booking systems
- Repository Pattern for data access abstraction
- Strategy Pattern for pluggable transfer agent routing
- Observer Pattern for real-time status notifications

Key Design Decisions

- Microservices over monolith for independent scaling of ingestion, aggregation, and booking services
- Kafka-based async pipeline to decouple order stages and handle PMS batch spikes
- Redis idempotency store to guarantee exactly-once semantics across retries
- Airflow scheduler for deterministic NSCC and RBC transmission windows
- Immutable event log as source of truth; CQRS read models for dashboard queries

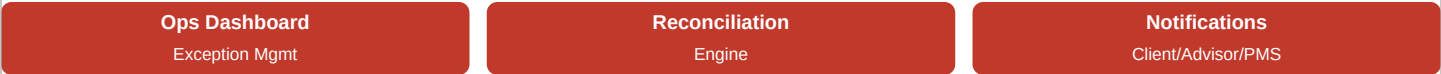
Architecture Overview

Immutable Audit Log (7-yr retention, FINRA Rule 4511) | TLS 1.2+, AES-256 | RBAC | 99.9% HA

INFRASTRUCTURE AND DATA LAYER



OUTPUTS AND OPERATIONS



3. Expected Time to Build a Prototype

Prototype Implementation Scope

While the entire MF-OMS is architected, the prototype implements three non-trivial end-to-end flows covering the core processing spine of the system.

1. **OMS State Machine.** Orders progress through states RECEIVED, VALIDATED, ENRICHED, AGGREGATED, TRANSMITTED, CONFIRMED, CONTRACTED, and BOOKED, with FAILED reachable at any stage. Every transition is immutably logged. REST APIs handle ingestion with duplicate detection and idempotency enforced at entry.
2. **NSCC Transmission and Confirmation.** At the 4:00 PM ET cutoff, ENRICHED orders are grouped by fund and trade date into a bulk order and transmitted to a simulated NSCC endpoint with exactly-once delivery via Redis idempotency keys. On confirmation, orders advance to CONFIRMED; mismatches become reconciliation breaks.
3. **Contract Processing and Trade Booking.** On receiving a contract (fund NAV and total shares), each client order gets a proportional share allocation. The OMS is updated and orders advance to CONTRACTED, then trade records are pushed to a booking service transitioning orders to BOOKED with channel notifications. A live order-state UI is a bonus if time permits.

Phase	Days	Ownership
Research and Planning	4	Domain analysis, API contracts, DB schema
System Design	5	Architecture, sequence flows, ADRs
Development	14	Dev 1-2: OMS + aggregation. Dev 3-4: NSCC + contracts. Dev 5: UI
Integration + Testing	5	Unit, integration, end-to-end across 3 flows
Refinement + Demo	3	Bug fixes, demo script, documentation

Total: ~31 days. Dev 1-2 own OMS and aggregation; Dev 3-4 own NSCC transmission and contracts; Dev 5 owns the UI. All five converge for integration and testing.

4. Domain

Finance

Sub-domain: Wealth Management and Capital Markets Operations

Industry Context

- US mutual fund industry manages over 25 trillion USD in assets (ICI 2024)
- NSCC processes millions of fund transactions daily across hundreds of member firms
- T+1 settlement mandate (SEC Rule 15c6-1, effective 2024) increases processing pressure
- Reconciliation failures carry direct penalties under FINRA regulations

Why This Fits the Course

This project exercises the full software architecture toolkit: heterogeneous workload management, event-driven design, exactly-once distributed processing, CQRS, and multi-stakeholder integration. The domain complexity ensures non-trivial architectural decisions with quantifiable trade-offs.

Industry Alignment

The architecture mirrors production systems at institutions such as JPMorgan, BlackRock, and Fidelity, making the learning directly applicable to fintech and capital markets engineering roles.